

Qrisc32

Five-stage RISC CPU

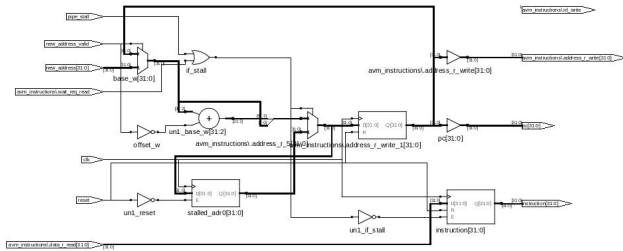
Viacheslav Vinogradov

Converted from `risc_present.odp`

- 5-stage RISC: IF, ID, EX, MEM, WB.
- AXI4-style instruction read master.
- AXI4-style data read/write master.
- 32 general-purpose 32-bit registers.
- Two EX-stage flags: Zero and Carry.

- Unconditional load to register.
- Unconditional store from register to memory.
- Unconditional jumps.
- Conditional relative jumps.
- ALU commands.
- Conditional load to register.

Instruction Fetch Stage



- Fetches instruction words from memory.
- Changes the program counter.
- Transfers PC value to the next stage for CALL and RETURN commands.

Instruction Decode Stage

- Decodes instruction words.
- Activates control signals for the next stage.
- Reads values from the standalone register file.
- Applies EX, MEM, and WB forwarding before driving the EX stage.

- Executes operations according to ID-stage control signals.
- Changes the SRC2 register by -4, -2, -1, 0, 1, 2, or 4.
- Forwards ALU results and SRC2 increments to the bypass network.
- Provides one of the writeback sources.

- Reads and writes memory.
- Contains an internal pipeline for AXI-style memory requests.
- Current repo contract uses single-beat, 32-bit aligned transfers.
- Read and write stages wait for their corresponding bus responses.

Writeback Stage

- Owns the architectural register-file commit path.
- Registers EX-stage and MEM-stage writeback sources.
- Writes EX/DST, EX/SRC2 increment, and MEM/DST results.
- Keeps deterministic priority when two sources target the same register.

- ID can obtain register values from EX, MEM, WB, or the register file.
- MEM forwarding has priority over EX forwarding, matching the original SV behavior.
- WB forwarding covers values committed one stage after EX or MEM.

- Branch-taken cost is 4 cycles.
- Advantage: simple IF stage.
- Disadvantage: large zero-slot delay.
- Efficient programs must account for the branch delay.

Instruction Format: LDR

31:28	27:26	25	24:22	21:15	14:10	9:5	4:0
LDR	type	off	incr src2	reserved	src2	src1	dst

- Type 0: $R_{dst} = R_{src1}$.
- Type 1: $R_{dst}[31:16] = code[20:5]$ (LDRH).
- Type 2: $R_{dst}[15:0] = code[20:5]$ (LDRL).
- Type 3: $R_{dst} = [R_{src1} + offset]$.

Instruction Format: STR

31:28	27:26	25	24:22	21:15	14:10	9:5	4:0
STR	type	off	incr src2	reserved	src2	src1	dst

- Stores Rdst to [Rsrc1 + offset].
- Syntax: STR Rdst, [Rsrc1 + offset], INCR2.

Instruction Format: Unconditional Jumps

31:28	27:26	25	24:22	21:15	14:10	9:5	4:0
JMP	type	off	offset/incr	code	src2	src1	dst

- Type 0: $pc[25:0] = code[25:0]$.
- Type 1: $pc = pc + offset$.
- Type 2: $pc = pc + offset, Rdst = pc$.
- Type 3: $pc = Rdst$.

Instruction Format: Conditional Jumps

31:28	27:26	25	24:22	21:15	14:10	9:5	4:0
JMPF	cond	off	offset	reserved	src2	src1	dst

- Type 0: $\text{JMPZ } \text{pc} = \text{pc} + \text{offset}.$
- Type 1: $\text{JMPNZ } \text{pc} = \text{pc} + \text{offset}.$
- Type 2: $\text{JMPC } \text{pc} = \text{pc} + \text{offset}.$
- Type 3: $\text{JMPNC } \text{pc} = \text{pc} + \text{offset}.$

Instruction Format: ALU

31:28	27:25	24:22	21:15	14:10	9:5	4:0
ALU	op	incr src2	reserved	src2	src1	dst

- 0: AND, 1: OR, 2: XOR, 3: ADD.
- 4: MUL, 5: shift left, 6: shift right, 7: compare.

Instruction Format: Conditional Load

31:28	27:26	25	24:22	21:15	14:10	9:5	4:0
LDRF	cond	off	incr src2	reserved	src2	src1	dst

- 0: LDRZ $R_{dst} = R_{src1}$ if Z is set, otherwise R_{src2} .
- 1: LDRNZ $R_{dst} = R_{src1}$ if Z is clear, otherwise R_{src2} .
- 2: LDRC $R_{dst} = R_{src1}$ if C is set, otherwise R_{src2} .
- 3: LDRNC $R_{dst} = R_{src1}$ if C is clear, otherwise R_{src2} .

Common Instruction Fields

31:28	27:26	25	24:22	21:15	14:10	9:5	4:0
opcode	type	off	incr src2	code/off	src2	src1	dst

- DST is the register that receives the operation result.
- SRC1 and SRC2 provide operation inputs.
- INCR SRC2 changes SRC2 by -4, -2, -1, 0, 1, 2, or 4.
- Off selects offset source: immediate code or signed SRC2.

Sorting Algorithms

- Shell sort.
- Modified shell sort.
- Bubble sort.
- Test data contains 10 integer words.

Shell Sort

```
ShellSort(int *a) {  
    int inc, i, j, temp;  
    for (inc = 5; inc > 0; inc = inc / 2) {  
        for (i = inc; i < 10; i += 1) {  
            temp = a[i];  
            for (j = i; (j >= inc) && (a[j-inc] > temp); j -= inc) {  
                a[j] = a[j-inc];  
            }  
            a[j] = temp;  
        }  
    }  
}
```

Modified Shell Sort

```
for (inc = 20; inc > 0; inc = inc / 2) {  
    for (i = inc; i < 40; i += 4) {  
        temp = a[i];  
        for (j = i; (j >= inc) && (a[j-inc] > temp); j = j - inc) {  
            a[j] = a[j-inc];  
        }  
        a[j] = temp;  
    }  
}
```

Bubble Sort

```
R0 = mem[00]; R1 = mem[04]; R2 = mem[08]; R3 = mem[0c]; R4 = mem[10];  
R5 = mem[14]; R6 = mem[18]; R7 = mem[1c]; R8 = mem[20]; R9 = mem[24];
```

```
R10 = R0;  
R11 = R0; // minimum  
if R1 < R11 then R11 = R1;  
...  
if R9 < R11 then R11 = R9;  
mem[0] = R11;
```

Sorting Performance

Algorithm	Reverse	Random	Sorted
Shell	842	612	176
Modified shell	712	669	519
Bubble	455	455	455

- Average cycles from the original slide: shell 543, modified shell 633, bubble 455.
- Bubble sort has the most stable runtime for this 10-word test.